IRVINE HACKS 2026
PRODUCT REQUIREMENTS DOCUMENT

HomeScore

AI-Powered Neighborhood & Property Recommendation Platform

Team Name:	[Your Team Name]
Hackathon:	IrvineHacks 2026
Track:	Best Use of AI in Real Estate — First American Title
Sponsor:	First American Title Insurance Company
Version:	2.0 — Updated February 2026

1. INTRODUCTION

1.1 Motivation / Background

Describe the problem you want to solve and why it is important. Max 300 words.

Buying a home involves two deeply intertwined decisions that most platforms treat as separate: choosing the right neighborhood and choosing the right house. Zillow and Redfin are built around house-first browsing — you filter by price and bedrooms and are then implicitly placed in whatever neighborhood those results happen to fall in. This is backwards. Research consistently shows that the neighborhood — its schools, character, density, walkability, and long-term trajectory — matters more to long-term satisfaction and financial return than most physical features of the house itself.

The second gap is temporal. Current platforms tell you what a home costs today. They do not tell you what it will cost to own over the next one, three, or five years — factoring in projected appreciation or depreciation driven by macroeconomic conditions, local market dynamics, and maintenance lifecycle. Buyers routinely underestimate total ownership costs by tens of thousands of dollars, leading to financial stress that surfaces only after closing.

HomeScore addresses both gaps with a four-stage pipeline. Stage one captures a detailed user preference profile covering both neighborhood character and physical house requirements. Stage two uses a trained classification model to score and rank all neighborhoods in the target city against those preferences, surfacing the best-fit communities before the buyer ever looks at a listing. Stage three uses hardcoded rule-based filtering to surface available homes in the selected neighborhood that match the user's physical house preferences. Stage four uses a trained regression model — or an LLM as a fallback — to project future home value and maintenance costs across multiple time horizons, incorporating current economic conditions. A single final LLM call synthesizes all outputs into an explainable, factor-by-factor summary that tells the buyer exactly why each recommendation was made.

1.2 State of the Art / Innovation

Describe how the problem is solved today and the innovation of your project. Max 200 words.

Existing consumer platforms are house-first and present-tense. Zillow's neighborhood data is static and generic — the same school rating shown to every buyer regardless of whether they have children. No platform performs neighborhood classification personalized to a specific user's declared preference profile. No consumer product provides multi-horizon cost projections that incorporate macroeconomic signals such as interest rate trends, regional employment growth, and housing inventory levels.

HomeScore innovates across three dimensions. First, it inverts the traditional search flow: neighborhood selection precedes house browsing, forcing the user to validate the community

before falling in love with a specific property. Second, it trains a real classification model on multi-source neighborhood data — not a simple lookup or scoring formula — producing a ranked neighborhood list that is unique to each user's preference vector. Third, it trains a regression model on historical transaction data augmented with macroeconomic features to project future value and maintenance costs with time-horizon specificity. The LLM is deployed only at the very end for narrative generation, with all predictive intelligence living in owned, trained models running at zero marginal inference cost. A practical LLM fallback is built in for the regression stage, acknowledging hackathon time constraints.

1.3 Project Goals

Describe the project general goals. Max 200 words.

1. Build a comprehensive user preference intake capturing neighborhood lifestyle factors (12+ dimensions) and physical house requirements (beds, baths, sqft, lot size, and more).
2. Train and deploy a neighborhood classification model that assigns a personalized neighborhood score to every neighborhood in the target city based on the user's preference vector.
3. Rank neighborhoods by score and allow the user to select one neighborhood to explore, then surface available homes in that neighborhood using hardcoded rule-based filtering against physical house preferences.
4. Train and deploy a regression model that projects future home value and maintenance costs over 6-month, 1-year, and 3-year horizons, incorporating macroeconomic and local market features. Provide a best-case and worst-case projection for each horizon.
5. Build an LLM fallback for the regression stage that uses current economic data passed via prompt to generate future cost and maintenance estimates when the trained model is not available.
6. Generate a single comprehensive LLM summary explaining: each factor that went into the neighborhood classification score (1-2 lines per factor), the house recommendation logic, and the future cost and maintenance projections.
7. Deliver a clean, demo-ready React frontend that walks the user through all four stages sequentially with clear visual feedback at each step.

1.4 Assumptions

Describe the assumptions you are making to solve the problem. Max 180 words.

HomeScore assumes access to publicly available neighborhood and property data sufficient to train the classification model and regression model. This includes Zillow Research data exports (historical median prices by zip code), US Census ACS data (demographics, income, commute), NCES school data, Walk Score API, Google Places API, and FRED macroeconomic data (interest rates, employment). No proprietary data sources are required.

The neighborhood classification model is trained on a labeled or proxy-labeled dataset where neighborhood quality scores are derived from combinations of public metrics. The regression model is trained on historical home transaction data augmented with macroeconomic features. Both models are trained offline before the hackathon and served from a FastAPI backend.

Available home listings in the selected neighborhood are retrieved from a publicly accessible source such as the Zillow API, Redfin data feeds, or RapidAPI real estate endpoints. If API access is constrained during the hackathon, a pre-fetched static dataset of current Orange County listings is used for the demo.

The LLM fallback for future cost projection is always available regardless of regression model status. The fallback receives structured economic data via prompt rather than relying on the model's training knowledge alone.

2. FOUR-STAGE USER FLOW

HomeScore is structured around four sequential stages. The user cannot skip ahead — each stage informs the next. This section describes the full user experience for each stage.

Stage 1: Preference Intake

The user begins by entering their target city (e.g., 'Irvine, CA'). They then complete a structured preference intake form divided into two sections: neighborhood preferences and physical house preferences. The full set of preference factors is documented in Section 3. The intake is designed to be completable in under 3 minutes and uses a mix of sliders, radio buttons, checkboxes, and a single free-text field for commute destination. All preference data is stored in a user session object that drives every downstream stage of the platform.

Stage 2: Neighborhood Classification & Ranked List

The backend classification model scores every neighborhood (zip code or census tract) in the target city. The model takes the user's neighborhood preference vector as input and outputs a Neighborhood Match Score (0-100) for each neighborhood. Neighborhoods are ranked by this score and displayed as a sorted list with the top match first. Each entry in the list shows the neighborhood name, overall match score, and a compact 3-4 metric summary (e.g., school rating, median price, walk score, safety index). The user selects one neighborhood from the ranked list to explore further. This is a deliberate single-selection step — the user commits to one community before seeing houses.

Stage 3: House Recommendations (Rule-Based Filtering)

Once a neighborhood is selected, the platform fetches available listings in that neighborhood from the listing data source. Hardcoded rule-based filtering logic is then applied using the user's physical house preferences to narrow the listings to matching homes. The filtering rules are fully deterministic — no model is involved. Results are displayed as a browsable list of cards sorted by a simple rule-based relevance score (described in Section 5). The user selects a specific house to evaluate.

Stage 4: Future Cost Projections & LLM Summary

For the selected house, the regression model (or LLM fallback) generates future value projections across three time horizons (6 months, 1 year, 3 years), each with a best-case and worst-case estimate. The model also projects maintenance costs over the same horizons. Finally, a single LLM API call receives the complete output of all three prior stages — preference profile, neighborhood score with per-factor breakdown, matched house details, and future cost projections — and generates a comprehensive plain-English summary explaining every recommendation made by the system.

3. PREFERENCE INTAKE SPECIFICATION

The preference intake is the foundation of the entire platform. All downstream models and filters consume this data. It is divided into two subsections: neighborhood preferences (inputs to the classification model) and physical house preferences (inputs to the rule-based filtering logic).

3.1 Neighborhood Preference Factors

These 12 factors are collected from the user and fed as a preference vector into the neighborhood classification model. Each factor is weighted by the user's stated priority level.

#	Factor	Input Type	Options / Range	Why It Matters
1	Area Type	Radio	Urban / Suburban / Rural	Determines density, walkability, lot sizes, and community character. A young professional wanting walkable nightlife and a retiree wanting quiet cul-de-sacs have fundamentally different needs.
2	Public School Quality	Slider	Priority: Low / Medium / High	For families with school-age children this is often the single most important factor. Drives use of NCES school rating data in classification model.
3	Commute Convenience	Text + Slider	Workplace address; Max acceptable commute: 15/30/45/60+ min	Enables computation of actual drive/transit time from neighborhood centroid to workplace via Google Maps API. Weighted heavily for in-office workers.
4	Safety / Crime Level	Slider	Priority: Low / Medium / High	Feeds FBI UCR or local crime index data into model. High-priority users receive strong negative weighting on neighborhoods with above-median crime rates.
5	Walkability & Amenities	Slider	Priority: Low / Medium / High	Walk Score API value for neighborhood. High priority for users who want to walk to coffee shops, restaurants, and errands. Lower priority for car-dependent suburban preferences.

6	Public Transit Access	Radio	Not needed / Helpful / Essential	Transit Score from Walk Score API. Essential for users without a car or commuting into a city center via rail or bus.
7	Family-Friendliness	Radio	Single / Couple / Family with kids / Empty nester	Composite proxy: school quality, park density (Google Places), playground proximity, traffic safety. Drives feature weighting in classification model.
8	Nightlife & Dining Scene	Slider	Priority: Low / Medium / High	Google Places POI count for restaurants, bars, entertainment venues within 1-mile radius of neighborhood centroid.
9	Green Space & Parks	Slider	Priority: Low / Medium / High	Google Places park count + proximity. Important for dog owners, families, and outdoor lifestyle users.
10	Neighborhood Stability & Growth	Radio	Prefer established / Prefer up-and-coming / No preference	Feeds appreciation trend data (Zillow ZHVI CAGR) and population growth (Census). Up-and-coming users prefer higher-growth, lower-current-price neighborhoods.
11	HOA Preference	Radio	Strongly prefer HOA / Neutral / Prefer no HOA	HOA presence affects community maintenance, aesthetics, rules, and monthly costs. Relevant to filtering neighborhoods by predominant HOA coverage rate.
12	Budget for Neighborhood Median	Range slider	Min and max median home price	Filters neighborhoods where the median home price falls within the user's budget range, preventing the model from recommending a neighborhood the user cannot afford.

3.2 Physical House Preference Factors

These factors are collected from the user and used exclusively by the hardcoded rule-based filtering logic in Stage 3. They are not inputs to the classification model.

#	Factor	Input Type	Options / Range	Filtering Behavior
1	Bedrooms	Stepper	1 – 6+	Hard filter: only homes with >= user minimum bedrooms are shown. If

				no homes match exactly, nearest available count is surfaced with a note.
2	Bathrooms	Stepper	1 – 5+ (0.5 increments)	Hard filter: only homes with >= user minimum bathrooms. Half-bath counted as 0.5.
3	Minimum Interior Sqft	Number input	500 – 10,000 sqft	Hard filter: listing sqft must be >= user minimum. If no exact matches, nearest results shown with delta labeled.
4	Maximum Interior Sqft	Number input	500 – 10,000 sqft	Hard filter: listing sqft must be <= user maximum to avoid showing homes far above space needs.
5	Lot Size Preference	Radio + Number	No preference / Min lot size in sqft	Soft filter: homes below minimum lot size are deprioritized and labeled 'Small lot' but not hidden entirely.
6	Property Type	Multi-select checkboxes	Single Family / Condo / Townhome / Multi-Family	Hard filter: only selected property types shown.
7	Maximum Listing Price	Number input	$100K – $5M+	Hard filter: only homes at or below this price shown. Forms the upper bound of the price range.
8	Minimum Listing Price	Number input	$0 – $5M	Soft filter: homes below this price flagged as potentially below expectations but not hidden.
9	Year Built Preference	Radio	No preference / After 1980 / After 2000 / After 2010	Soft filter: homes outside preference flagged with year built label. Relevant to maintenance cost estimation.
10	Garage	Radio	Not needed / 1-car / 2-car / 3+ car	Soft filter: homes without requested garage size deprioritized and labeled accordingly.
11	Pool	Radio	No preference / Preferred / Required	Soft filter (preferred) or hard filter (required). Pool presence from listing data.
12	Stories	Radio	No preference / Single-story / Multi-story	Soft filter: relevant for elderly buyers or buyers with mobility considerations.

4. NEIGHBORHOOD CLASSIFICATION MODEL

The neighborhood classification model is the core ML artifact of Stage 2. It takes a neighborhood's objective feature vector and a user's preference weight vector as inputs, and outputs a Neighborhood Match Score (0-100) representing how well that neighborhood fits that specific user. The model is trained offline and served at zero marginal inference cost from the FastAPI backend.

4.1 Model Type & Framing

Attribute	Detail
Model Type	Random Forest Classifier trained to classify neighborhoods into quality tiers (Top Match / Good Match / Moderate Match / Poor Match) for a given user preference profile. A regression variant (Random Forest Regressor) outputs a continuous 0-100 score directly.
Why Random Forest	Handles mixed feature types (numeric metrics + categorical preference flags) without normalization. Produces feature importances that feed directly into the LLM summary. Fast inference on CPU. Robust to the moderate dataset sizes available from public sources.
Personalization Approach	The user's preference weight vector is incorporated by multiplying each neighborhood feature by its corresponding user priority weight before feeding into the model. This means the same model serves all users — no per-user retraining required.
Output	Continuous Neighborhood Match Score (0-100) per neighborhood per user. Feature importance values per neighborhood feature are also extracted and passed to the LLM for explanation generation.
Inference Cost	< 1ms per neighborhood on CPU. A city with 30 zip codes is scored in under 30ms total. Zero API cost.

4.2 Neighborhood Feature Set

Feature	Data Source	Description
Median Home Price	Zillow Research ZHVI CSV	Current median home value for the zip code. Matched against user's budget range.

School Rating (Avg)	NCES Common Core of Data + GreatSchools API	Average rating of public schools within the zip code boundary. Scored 1-10.
Crime Index	FBI UCR / local PD open data / Neighborhood Scout public data	Composite violent and property crime rate per 1,000 residents. Lower is better.
Walk Score	Walk Score API	Walkability score 0-100 for neighborhood centroid address.
Transit Score	Walk Score API	Public transit availability score 0-100.
Commute Time to Workplace	Google Maps Distance Matrix API	Median drive time in minutes from neighborhood centroid to user's entered workplace address.
Restaurant & Bar Density	Google Places API	Count of restaurants and bars within 1-mile radius of neighborhood centroid.
Park & Green Space Count	Google Places API	Count of parks and recreational areas within 1-mile radius.
Area Type Classification	Census TIGER/Line + population density	Urban / Suburban / Rural classification derived from Census tract population density.
5-Year Price Appreciation CAGR	Zillow Research ZHVI CSV	Compound annual growth rate of median home value over past 5 years. Proxy for neighborhood trajectory.
Median Household Income	Census ACS 5-Year Estimates	Median household income for the zip code. Correlates with neighborhood stability and service quality.
Population Growth Rate	Census ACS multi-year comparison	Population change percentage over 5 years. Positive growth indicates desirable area.
HOA Coverage Rate	ATTOM Data / county assessor	Percentage of properties in the zip code with active HOA. Proxy for HOA prevalence.
Hospital Proximity	Google Places API	Distance in miles to nearest hospital from neighborhood centroid.
Grocery Store Proximity	Google Places API	Distance in miles to nearest full-service grocery store.

4.3 Training Data & Methodology

Attribute	Detail
Primary Dataset	Zillow Research ZHVI by zip code (free CSV download). Provides historical median prices for all zip codes nationally. Orange County: ~30 zip codes with 10+ years of monthly data.
Secondary Datasets	Census ACS 5-Year Estimates (free API). NCES Common Core (free bulk download). Walk Score API (free tier). Google Places API (free tier quota). FBI UCR crime data (free download). FRED macroeconomic data (free API).
Labeling Strategy	Because direct 'neighborhood quality' labels do not exist in any public dataset, a proxy labeling approach is used. Neighborhoods are ranked by a composite of objective metrics (school rating, crime rate, appreciation CAGR, walk score, income) and binned into four quality tiers. These proxy labels serve as the classification target during training. This is a standard approach in real-world neighborhood scoring systems.
Training Geography	Orange County, CA zip codes for hackathon demo. Architecture generalizable to any US metro with the same data sources.
Validation	5-fold cross-validation. Target: classification accuracy > 80% on proxy-labeled tiers. Feature importances extracted and validated for economic sensibility (school rating and crime index should rank as top features for most user profiles).
Hyperparameter Tuning	GridSearchCV on n_estimators, max_depth, min_samples_split. Optimized for F1-macro score across four tiers.

4.4 Feature Importances for LLM Explanation

After scoring a neighborhood for a specific user, the model's feature_importances_ array is extracted and filtered to the top 5 most influential features for that scoring. These are passed to the LLM along with the raw values, enabling the LLM to generate a factor-by-factor explanation. For example: 'School Rating (weight: 0.31): The schools in this neighborhood average a 8.2/10 rating, which strongly aligns with your high priority for public education.' This per-factor explanation is a key differentiator of HomeScore's LLM summary and is described further in Section 6.

5. HOUSE RECOMMENDATION LOGIC (RULE-BASED)

Stage 3 of HomeScore does not use a machine learning model. House recommendations within the selected neighborhood are generated entirely by hardcoded deterministic filtering and ranking rules applied to available listings fetched from a public data source. This is a deliberate architectural choice: matching a house to explicit physical preferences is a constraint satisfaction problem, not a prediction problem, and rule-based logic is more transparent, more reliable, and easier to explain than a model for this task.

5.1 Listing Data Source

Source	Access Method	Data Provided	Fallback
Zillow API (Bridge Interactive)	REST API — requires approval but free for developers	Active listings with price, beds, baths, sqft, lot size, property type, year built, photos, address	Pre-fetched static JSON dataset of Orange County listings
RapidAPI Real Estate APIs (Zillow / Redfin scrapers)	REST API — paid per call (~$0.01/call)	Same fields as above; faster approval than official Zillow API	Same static fallback dataset
Redfin Data Center	CSV download by geography	Historical and current listing data; less real-time but reliable for demo	Same static fallback dataset

5.2 Filtering Rules (Applied Sequentially)

Hard Filters — Listings Excluded If Not Met

- Price: listing_price <= user.max_price
- Bedrooms: listing.beds >= user.min_beds
- Bathrooms: listing.baths >= user.min_baths
- Interior sqft: listing.sqft >= user.min_sqft AND listing.sqft <= user.max_sqft (if max provided)
- Property type: listing.type IN user.selected_property_types
- Pool (if 'Required'): listing.has_pool == true

Soft Filters — Listings Kept But Labeled

- Lot size below preference: listing flagged with 'Smaller lot than preferred' badge

- Year built outside preference: listing flagged with 'Older build' or 'Newer than expected' badge
- Garage below preference: listing flagged with 'Fewer garage spaces than preferred' badge
- Stories mismatch: listing flagged with 'Multi-story' or 'Single-story' note
- Price below user minimum: listing flagged with 'Below your stated minimum — verify condition' badge

5.3 Relevance Ranking Rule

After filtering, remaining listings are ranked by a simple additive relevance score. Each soft filter that is satisfied adds points; each hard filter satisfied at a higher margin than required adds bonus points. The formula is deterministic and fully explainable:

Rule	Points Awarded
Beds exactly match user preference (not just minimum met)	+ 15 points
Baths exactly match user preference	+ 10 points
Sqft within 10% of user's ideal midpoint (avg of min and max)	+ 15 points
Lot size meets or exceeds user preference	+ 10 points
Year built within user's preferred era	+ 10 points
Pool present (when preferred but not required)	+ 10 points
Garage matches user preference	+ 10 points
Stories match user preference	+ 10 points
Price is 5-15% below user maximum (value zone)	+ 10 points
Zero soft filter flags (perfect match on all dimensions)	+ 10 bonus points

The top-ranked listings (up to 10) are displayed in the home browsing view. The relevance score and the number of flags are shown on each card so the user understands the ranking.

6. FUTURE COST REGRESSION MODEL & LLM FALLBACK

Stage 4 of HomeScore generates two sets of projections for the selected house: future home value projections (appreciation or depreciation) and maintenance cost projections. Both are computed across three time horizons — 6 months, 1 year, and 3 years — with a best-case and worst-case estimate for each horizon. The primary mechanism is a trained regression model. A practical LLM-based fallback is available if the regression model is not ready by demo time.

6.1 Future Value Regression Model (Primary)

Attribute	Detail
Model Type	XGBoost Regressor. Predicts future home value at each time horizon as a percentage change from current listing price. Three separate model instances: one per time horizon (6-month, 1-year, 3-year).
Target Variable	Percentage change in home value from sale date to T+horizon months. Computed from historical transaction pairs in the training dataset.
Property Features	Current listing price, sqft, beds, baths, year built, property type, lot size, days on market, current neighborhood median price.
Neighborhood Features	5-year historical price CAGR (Zillow ZHVI), current months of inventory for zip code (Zillow market data), median days on market for zip code, year-over-year price change for zip code.
Macroeconomic Features	30-year fixed mortgage rate at time of sale (FRED MORTGAGE30US series), Federal Funds Rate (FRED FEDFUNDS), Case-Shiller Home Price Index for metro (FRED), regional unemployment rate (BLS), new housing permits in county (Census Building Permits Survey), consumer confidence index (FRED).
Best-Case / Worst-Case	For each horizon, the model is run twice: once with optimistic macroeconomic feature values (rates declining, inventory tightening, confidence rising) and once with pessimistic values (rates rising, inventory expanding, confidence falling). These produce the best-case and worst-case projected values displayed to the user.
Training Dataset	Historical home sales with known future prices at each horizon. Source: county assessor transaction records + Zillow Research historical transaction data. Target: ~50,000 Orange County transactions over 10 years providing sufficient paired observations at each horizon.

Validation	Temporal train/test split: train on transactions before 2022, test on 2022-2024 transactions. Evaluate MAPE per horizon. Target: < 10% MAPE on 6-month horizon, < 14% on 1-year, < 18% on 3-year (accuracy naturally decreases with horizon length).
Inference Cost	< 2ms per prediction on CPU. Zero API cost.

6.2 Maintenance Cost Projection

Attribute	Detail
Model Type	Rule-based regression model calibrated on industry cost databases. Computes expected maintenance costs by year based on home age, size, and construction type. No ML training required for this component.
Data Sources	NAHB home component lifecycle data (publicly available): average lifespans for roof (20-30yr), HVAC (15-20yr), water heater (8-12yr), plumbing (20-50yr by material), electrical panel (30-40yr), exterior paint (5-10yr), appliances (10-15yr). RSMeans Residential Cost Data and HomeAdvisor True Cost Guide: regional average replacement costs by component and square footage.
Logic	For each major system, compute: component_age = current_year - (install_year or year_built). Replacement probability within horizon = f(component_age / expected_lifespan). Expected cost within horizon = replacement_probability * replacement_cost * regional_cost_multiplier. Annual base maintenance = 1.0-1.5% of listing price (industry standard), adjusted by condition score if available.
Output	Total projected maintenance cost over each horizon (6mo, 1yr, 3yr), itemized by component. High-risk components (replacement probability > 70% within horizon) flagged with estimated replacement cost range. Best-case (no major failures) and worst-case (one or more high-risk components fail within horizon) estimates provided.
Inference Cost	Pure computation. Zero API cost. < 5ms runtime.

6.3 LLM Fallback for Future Cost Projections

If the regression model is not trained and deployed by demo time, a structured LLM fallback is used in its place. This fallback does not rely on the LLM's training knowledge alone — it passes current economic data as structured context in the prompt, ensuring the output is grounded in real conditions rather than stale training data.

Fallback Prompt Structure

The fallback prompt passes the following structured data to the LLM:

- Property: address, listing price, sqft, year built, property type, neighborhood zip code
- Current market conditions: current 30-year mortgage rate (fetched from FRED API in real time), current months of inventory for the zip code (Zillow market data), year-over-year price change for the zip code (Zillow ZHVI)
- Macroeconomic context: current Federal Funds Rate, recent CPI trend, regional unemployment rate, local new construction permit trend
- Instruction: 'Based on the above property details and current economic conditions, provide best-case and worst-case projected home values at 6 months, 1 year, and 3 years from today. Also estimate total maintenance costs for each horizon given the home age and size. Provide specific dollar figures with reasoning. Do not use general knowledge — base your estimates on the economic data provided above.'

The fallback output is clearly labeled in the UI as 'AI-estimated projection' rather than 'Model-generated projection' so users understand the distinction. The same economic data passed to the fallback LLM is also included in the final summary LLM call for consistency.

7. LLM SUMMARY GENERATION

After all four stages are complete, a single LLM API call generates the comprehensive HomeScore Summary. This summary is the primary user-facing output of the platform and must be detailed, structured, and fully explainable. The LLM receives a compact JSON payload containing all model outputs and generates a structured narrative organized into four sections.

7.1 LLM Input Payload Structure

The following structured JSON is passed to the LLM. Total payload is designed to stay under 1,500 tokens to minimize cost:

Payload Field	Content	Source
user_profile	Neighborhood preference priorities (factor name + priority level) and physical house preferences summary	Stage 1 intake
neighborhood_score	Overall Neighborhood Match Score (0-100) for selected neighborhood	Classification model
neighborhood_factors	Array of top 5 feature names, their raw values, and their feature importance weights from the classification model	Classification model feature_importances_
neighborhood_name	Name and zip code of selected neighborhood	Stage 2 selection
selected_house	Address, listing price, beds, baths, sqft, year built, property type, days on market	Stage 3 selection
house_match_score	Relevance score and list of satisfied/unsatisfied soft filter flags	Rule-based filter output
future_value_projections	Object: {6mo: {best, worst}, 1yr: {best, worst}, 3yr: {best, worst}} in dollar amounts and percentage change	Regression model or LLM fallback
maintenance_projections	Object: {6mo: {total, flagged_items}, 1yr: {total, flagged_items}, 3yr: {total, flagged_items}}	Maintenance estimator
economic_context	Current mortgage rate, zip CAGR, months of inventory, Fed Funds Rate, regional unemployment	FRED API + Zillow
projection_source	'regression_model' or 'llm_fallback' — tells the summary LLM which method generated cost projections	Backend flag

7.2 LLM Summary Output Structure

The LLM is instructed via system prompt to generate a summary in exactly four sections:

Section A: Neighborhood Score Explanation

One introductory sentence stating the overall Neighborhood Match Score and what it means. Then, for each of the top 5 factors that drove the neighborhood score, exactly 1-2 sentences explaining: what the raw value of that factor is, how it was weighted given the user's stated priorities, and whether it positively or negatively contributed to the score. Example output:

> *Woodbridge scored 84/100 as a neighborhood match for your profile.*
>
> *School Rating (High Priority): Public schools in Woodbridge average 8.6/10 on GreatSchools — one of the highest in Orange County. Given your high priority for school quality, this was the strongest positive contributor to your neighborhood score.*
>
> *Commute Time (High Priority): The estimated drive from Woodbridge to your workplace in Irvine Business Complex is 11 minutes, well within your stated maximum of 30 minutes. This contributed strongly to your score.*
>
> *Crime Index (Medium Priority): Woodbridge has a crime index of 18 per 1,000 residents, below the Orange County median of 24. Given your medium priority for safety, this was a moderate positive contributor.*
>
> *Area Type (High Priority): Woodbridge is classified as Suburban, which matches your stated preference. This contributed positively to your score.*
>
> *Walkability (Low Priority): Woodbridge has a Walk Score of 52, which is below average. Because you marked walkability as low priority, this had only a minor negative effect on your score.*

Section B: House Recommendation Explanation

2-3 sentences explaining why this specific house was surfaced as a top match, referencing the relevance score, which hard filters it met, and which (if any) soft filter flags apply. The explanation should directly connect the user's stated physical preferences to the house's features.

Section C: Future Value Projection Explanation

For each time horizon (6 months, 1 year, 3 years), 2-3 sentences explaining: the best-case and worst-case projected value, what economic factors drive the best case, and what economic factors drive the worst case. The LLM should name specific economic signals from the payload (e.g., 'the current 30-year mortgage rate of 6.8%' or 'Woodbridge's 5-year appreciation CAGR of 4.2%'). If the projection source is 'llm_fallback', the summary should note this clearly.

Section D: Maintenance Cost Summary

A concise explanation of projected maintenance costs across the three horizons. Any high-risk components (roof, HVAC, etc.) are named explicitly with their estimated replacement cost range and the reasoning for flagging them (e.g., 'The home was built in 1987, making its HVAC system approximately 37 years old — well past the typical 15-20 year lifespan, with a replacement cost estimated at $8,000-$14,000').

7.3 LLM API Configuration

Parameter	Value	Rationale
Model	claude-sonnet-4-6 or gpt-4o	Both produce well-structured, reliable outputs for structured summarization tasks
Max tokens (input)	~1,500 tokens	Compact payload keeps cost low; all fields are pre-summarized before passing
Max tokens (output)	800 tokens	Sufficient for four-section summary without excessive verbosity
Temperature	0.3	Low temperature for factual, consistent output. Not creative writing — precision matters.
System prompt role	'You are a real estate financial analyst explaining property recommendations to a first-time homebuyer. Be specific, cite the data provided, and never add information not present in the input payload.'	Grounds the LLM firmly in the structured data and prevents hallucination
Estimated cost per call	~$0.02-0.04 USD	Well within acceptable hackathon cost budget
Calls per user session	1 (for summary) + 1 (if fallback active for projections) = max 2	Cost per full user session < $0.08

8. SYSTEM ARCHITECTURE OVERVIEW

8.1 Component Diagram

Layer	Component	Responsibility
Frontend	React + Vite + TypeScript	Four-stage sequential UI. Stage 1: preference intake form. Stage 2: neighborhood ranked list. Stage 3: house browsing cards with filter badges. Stage 4: future cost projections dashboard + LLM summary panel.
Frontend	Tailwind CSS + Shadcn/UI	Styling and component library. Consistent with winning HomeSuite stack.
Frontend	Mapbox GL JS	Optional: map view of ranked neighborhoods overlaid with score heatmap.
Frontend	Recharts	Future value projection charts — line charts showing best/worst case trajectories over three horizons.
API Gateway	FastAPI (Python)	Orchestrates all backend services. Key endpoints: POST /preferences, GET /neighborhoods/{city}, GET /homes/{zip_code}, POST /project/{listing_id}, POST /summary.
ML Service 1	Neighborhood Classification Model	Scikit-learn Random Forest served via joblib. Scores all neighborhoods in target city against user preference vector. Returns ranked list with per-neighborhood feature importances.
ML Service 2	Future Value Regression Model	XGBoost models (3 instances: 6mo, 1yr, 3yr) served via joblib. Returns best-case and worst-case value projections for a specific listing.
ML Service 3	Maintenance Cost Estimator	Deterministic rule engine. Returns itemized maintenance cost projections with flagged high-risk components.
LLM Fallback	Future Cost LLM Fallback	Claude or GPT-4o API call with structured economic context prompt. Activated when regression model unavailable. Returns structured JSON matching regression model output format.
LLM Summary	HomeScore Summary Generator	Single Claude or GPT-4o API call receiving full session context. Returns four-section structured narrative.

Data Layer	FRED API	Real-time macroeconomic data: mortgage rates, Fed Funds Rate, Case-Shiller index. Called fresh for each session to ensure current economic context.
Data Layer	Zillow Research CSVs	Offline: historical ZHVI data for model training and zip-level CAGR computation. Online: current months of inventory and median days on market per zip.
Data Layer	Google Places API	POI proximity data for neighborhood feature computation.
Data Layer	Walk Score API	Walkability and transit scores per neighborhood.
Data Layer	Census ACS API	Neighborhood demographics, income, population growth.
Data Layer	NCES / GreatSchools	School ratings by geographic area.
Data Layer	Listing API (Zillow Bridge / RapidAPI)	Active home listings in selected neighborhood with all physical attributes.
Database	PostgreSQL	Caches neighborhood feature vectors, pre-computed neighborhood scores per city, API responses. Listing data cached after first fetch.

8.2 Request Flow Sequence

Step	Actor	Action
1	User → Frontend	Submit city + completed preference intake form
2	Frontend → FastAPI	POST /preferences { city, neighborhood_prefs, house_prefs }
3	FastAPI → PostgreSQL	Check cache for pre-computed neighborhood features for city
4	FastAPI → External APIs	If not cached: fetch Walk Score, Google Places, Census ACS, Zillow ZHVI for all zip codes in city
5	FastAPI → Classification Model	Score all neighborhoods against user preference vector. Return ranked list with feature importances.
6	FastAPI → Frontend	Return ranked neighborhood list with scores and key metrics
7	User → Frontend	Select one neighborhood from ranked list
8	FastAPI → Listing API	Fetch active listings in selected neighborhood zip code

9	FastAPI → Rule Engine	Apply hard and soft filters to listings using house preferences. Compute relevance scores. Return ranked listing results.
10	FastAPI → Frontend	Return filtered, ranked listing cards
11	User → Frontend	Select one listing for full evaluation
12	FastAPI → FRED API	Fetch current mortgage rate, Fed Funds Rate, CPI trend
13	FastAPI → Regression Model	Run 6mo/1yr/3yr value projections with best/worst-case economic scenarios [OR fallback: LLM API call with economic context]
14	FastAPI → Maintenance Estimator	Run maintenance cost projection for selected listing
15	FastAPI → LLM API	POST summary prompt with full session context (~1,500 tokens). Return four-section HomeScore Summary.
16	FastAPI → Frontend	Return complete Stage 4 payload: projections + summary
17	Frontend → User	Render future cost charts and LLM summary panel

9. REQUIREMENTS

9.1 Use Cases

Use Case 1: Complete Preference Intake

Actors: First-time homebuyer, HomeScore System

Precondition: User has a target city in mind and a general sense of their lifestyle and housing needs.

Flow — Basic Path:

- User enters target city. System validates city and resolves to supported metro area.
- User completes neighborhood preference section: 12 factors with sliders, radio buttons, and checkboxes. Commute destination optionally entered.
- User completes physical house preference section: beds, baths, sqft range, lot preference, property type, price range, year built, garage, pool, stories.
- System stores preference object in session. Constructs neighborhood preference weight vector.
- System proceeds to neighborhood scoring.

Alternative Path: User skips commute destination — commute dimension excluded from neighborhood score with user notified.

Postcondition: Complete preference object in session. Ready for classification model.

Use Case 2: View & Select From Ranked Neighborhood List

Actors: Homebuyer, Classification Model, HomeScore System

Flow — Basic Path:

- Classification model scores all neighborhoods in city. Results sorted descending by match score.
- Ranked list displayed with: neighborhood name, overall score badge, and 3-4 key metrics (school rating, median price, walk score, safety index).
- User can expand any list item to see all 15 feature values for that neighborhood.
- User selects one neighborhood. System proceeds to listing fetch.

Postcondition: One neighborhood selected and locked. House browsing view loads.

Use Case 3: Browse Rule-Filtered House Listings

Actors: Homebuyer, Rule Engine, Listing API

Flow — Basic Path:

- System fetches active listings in selected neighborhood. Hard filters applied first — non-matching listings removed.

- Soft filters applied — surviving listings receive flag badges for any unmet soft preferences.
- Listings ranked by relevance score. Top 10 displayed as browsable cards.
- Each card shows: photo, address, price, beds/baths/sqft, relevance score, and flag count.
- User selects one listing for full evaluation.

Alternative Path: Fewer than 3 listings pass hard filters — system notifies user and suggests relaxing one constraint (e.g., expand sqft range by 10%).

Postcondition: One listing selected. System proceeds to Stage 4 projections.

Use Case 4: Generate Future Cost Projections

Actors: HomeScore System, Regression Model (or LLM Fallback), FRED API

Flow — Basic Path (Regression Model):

- FastAPI fetches current economic indicators from FRED API.
- Regression model runs 6 predictions: {6mo best, 6mo worst, 1yr best, 1yr worst, 3yr best, 3yr worst}.
- Maintenance estimator computes costs for same three horizons.
- Results returned with projection source labeled as 'regression_model'.

Flow — Alternative Path (LLM Fallback):

- FastAPI fetches current economic indicators from FRED API.
- LLM API called with structured prompt including property details + economic context.
- LLM returns projected values in same JSON format as regression model.
- Results returned with projection source labeled as 'llm_fallback'.

Postcondition: Six future value projections and three maintenance cost projections available for LLM summary.

Use Case 5: Generate & Display LLM HomeScore Summary

Actors: HomeScore System, LLM API, Homebuyer

Precondition: All Stage 1-4 outputs available in session.

Flow:

- FastAPI assembles compact JSON payload from all session data (~1,500 tokens).
- Single LLM API call with structured system prompt and session payload.
- LLM returns four-section summary: neighborhood score explanation (per-factor), house match explanation, future value explanation, maintenance summary.
- Frontend renders summary in a dedicated panel with each section clearly labeled and collapsible.

Postcondition: User has a complete, explainable summary of every recommendation made by HomeScore.

Use Case 6: Relax Constraints When No Houses Match

Actors: Homebuyer, Rule Engine

Flow:

- Rule engine returns fewer than 3 listings after hard filtering.
- System identifies which hard filter is most restrictive (eliminates the most listings).
- User presented with: 'No homes match all your requirements. Relaxing your minimum sqft from 2,000 to 1,800 would surface 7 additional homes. Would you like to adjust?'
- If user accepts, filter is relaxed and listings re-fetched.

9.2 User Stories

User Story 1 — Neighborhood-First Decision Making

As a buyer relocating to a new city, I want to understand which neighborhoods fit my lifestyle before I look at any specific houses, so that I don't fall in love with a home in a community that doesn't match my needs.

Acceptance Criteria:

- Neighborhood ranked list is generated within 5 seconds of preference submission.
- Each neighborhood entry shows the overall match score and the single strongest positive factor for that user.
- User cannot access house listings until a neighborhood is explicitly selected.

User Story 2 — Transparent AI Recommendations

As a skeptical buyer, I want to understand exactly why HomeScore recommended this neighborhood and this house, with specific data points cited, so that I can validate the recommendations myself.

Acceptance Criteria:

- LLM summary includes a 1-2 sentence explanation for each of the top 5 neighborhood score factors.
- Each explanation cites the actual raw value of the factor (e.g., 'school rating of 8.6/10') not just a generic statement.
- House match explanation lists exactly which filters were met and which flags apply.

User Story 3 — Understanding True Long-Term Cost

As a buyer at my budget ceiling, I want to see the projected total cost of owning this home over 3 years — including maintenance — in both a best-case and worst-case scenario, so that I can plan financially rather than be surprised after closing.

Acceptance Criteria:

- Future value projections shown for all three horizons with best and worst case labeled distinctly.
- Maintenance cost summary identifies any flagged high-risk components by name with estimated replacement cost.

- A 'Total Projected Ownership Cost' figure is shown combining projected mortgage payments + maintenance estimate for each horizon.

User Story 4 — LLM Fallback Transparency

As a user receiving future cost projections, I want to know whether the projections came from a trained model or an AI estimate, so that I can calibrate how much weight to give the numbers.

Acceptance Criteria:

- Projection cards are labeled either 'Model Projection' (regression model) or 'AI Estimate' (LLM fallback).
- LLM summary explicitly states in Section C whether projections are model-generated or AI-estimated.
- If LLM fallback, the specific economic indicators used as inputs are listed in the summary.

10. CODING AND TESTING

10.1 Repository

GitHub: [Repository link — e.g., github.com/[team]/homescore]

10.2 Testing Strategy

Classification Model Testing

The neighborhood classification model is validated using 5-fold cross-validation on the proxy-labeled neighborhood dataset. Primary metric: F1-macro across four quality tiers. Secondary metric: mean Neighborhood Match Score difference between user profiles that should produce different rankings (e.g., 'Family with kids prioritizing schools' vs. 'Single professional prioritizing walkability' on the same set of neighborhoods should produce materially different ranked lists). Feature importance validation: school rating and crime index should consistently rank in the top 3 features for most user profiles.

Rule-Based Filter Testing

The filtering logic is tested with a manually curated dataset of 20 synthetic listings and 5 synthetic user preference profiles. Each test case specifies the expected set of listings that should pass hard filters, the expected relevance ranking order, and the expected soft filter badges. Because the logic is deterministic, tests are exact — there is no probabilistic tolerance. Edge cases tested: no listings match hard filters (constraint relaxation flow triggered), all listings pass all soft filters (perfect match scenario), single listing available (minimal choice scenario).

Regression Model Testing

Future value regression models are evaluated using temporal train/test split (train pre-2022, test 2022-2024). MAPE computed per horizon. Best-case / worst-case spread validated to ensure the economic scenario perturbation produces a meaningful and realistic range (spread should be 8-20% of home value for 3-year horizon based on historical volatility). Macroeconomic feature sensitivity tested by holding all features constant and varying mortgage rate alone — model output should move in the expected direction (higher rates → lower appreciation).

LLM Summary Testing

The LLM summary is tested against a set of 5 synthetic session payloads representing diverse user profiles and property scenarios. For each payload, the generated summary is evaluated against a rubric: Does Section A include exactly 5 factor explanations? Does each explanation cite the raw value? Does Section C reference specific economic indicators from the payload? Does the summary avoid introducing facts not present in the payload (hallucination check)?

10.3 Testing Metrics

Component	Metric	Target
Classification Model	F1-macro on proxy-labeled neighborhoods	≥ 0.80
Classification Model	Ranking divergence between contrasting profiles	Top 3 neighborhoods differ by ≥ 2 positions
Rule Filter Engine	Exact match on all 20 curated test cases	100%
Regression Model (6-month)	MAPE on 2022-2024 hold-out	< 10%
Regression Model (1-year)	MAPE on 2022-2024 hold-out	< 14%
Regression Model (3-year)	MAPE on 2022-2024 hold-out	< 18%
LLM Summary — Section A	Factor explanation count	Exactly 5 factors
LLM Summary — hallucination	Facts cited not in payload	0 instances
End-to-end Stage 4	Total response time (projections + summary)	< 10 seconds
LLM cost per session	USD (max 2 LLM calls)	< $0.08

11. SYSTEM MODELS

11.1 UML Class Diagram

Class	Key Attributes	Key Methods
UserSession	sessionId, city, neighborhoodPrefs: NeighborhoodPrefs, housePrefs: HousePrefs, preferenceWeightVector: Dict	buildWeightVector(), toJSON()
NeighborhoodPrefs	areaType, schoolPriority, commuteDestination, maxCommute, safetyPriority, walkabilityPriority, transitNeed, familyProfile, nightlifePriority, greenSpacePriority, stabilityPref, hoaPref, budgetMin, budgetMax	validate(), toWeightVector()
HousePrefs	minBeds, minBaths, minSqft, maxSqft, lotSizePref, propertyTypes: List, maxPrice, minPrice, yearBuiltPref, garagePref, poolPref, storiesPref	toFilterRules(), toJSON()
NeighborhoodFeatureVector	zipCode, name, medianPrice, schoolRating, crimeIndex, walkScore, transitScore, commuteMinutes, restaurantCount, parkCount, areaType, appreciationCAGR, medianIncome, populationGrowth, hoaCoverageRate, hospitalDist, groceryDist	fromAPIs(), toModelInput(weightVector)
ClassificationModel	model: RandomForestClassifier, featureNames: List	scoreNeighborhood(featureVector) -> float, getImportances() -> Dict, rankNeighborhoods(List[NeighborhoodFeatureVector]) -> List
ListingRecord	listingId, address, price, beds, baths, sqft, lotSqft, propertyType, yearBuilt, hasPool, garageSpaces, stories, photos: List	fromAPI(), toFilterInput()

RuleFilterEngine	housePrefs: HousePrefs	applyHardFilters(listings) -> List, applySoftFilters(listings) -> List[{listing, flags}], computeRelevanceScore(listing) -> int, suggestConstraintRelaxation()
RegressionModel	models: Dict[horizon, XGBoostRegressor], featureNames: List	projectValue(listing, economicContext, scenario) -> float, projectAllHorizons(listing, economicContext) -> Dict
MaintenanceEstimator	lifespanLookup: Dict, costLookup: Dict	estimate(listing) -> {horizons: Dict, flaggedItems: List}
LLMFallback	client: LLM_API, promptTemplate: String	projectFutureCost(listing, economicContext) -> Dict
LLMSummaryGenerator	client: LLM_API, systemPrompt: String	generateSummary(sessionPayload) -> {sectionA, sectionB, sectionC, sectionD}
HomeScoreSession	userSession, rankedNeighborhoods, selectedNeighborhood, filteredListings, selectedListing, projections, summary	assemble(), toLLMPayload()

11.2 UML State Diagram: User Session

State	Entry Trigger	Exit Trigger	Key Activity
CITY_INPUT	App launch	City validated	User enters city. System validates against supported metros.
PREFERENCE_INTAKE	City validated	All required fields completed	User completes neighborhood + house preference forms. Weight vector built.
NEIGHBORHOOD_SCORING	Intake submitted	Scores returned	Classification model runs on all neighborhoods in city. Ranked list generated.
NEIGHBORHOOD_SELECTION	Scores returned	User selects one neighborhood	User reviews ranked list. Expands entries. Selects one neighborhood.

LISTING_FETCH	Neighborhood selected	Listings returned and filtered	Listing API called. Rule engine applies hard then soft filters. Relevance scores computed.
HOUSE_BROWSING	Listings filtered	User selects one listing	User browses cards. Reviews flags. Selects one property for evaluation.
PROJECTION_RUNNING	Listing selected	Projections returned	FRED API fetched. Regression model (or LLM fallback) runs. Maintenance estimator runs.
SUMMARY_GENERATING	Projections returned	Summary returned	LLM API called with full session payload. Four-section summary generated.
RESULTS_VIEW	Summary returned	User starts over or exits	User reviews projections, charts, and LLM summary. Can restart flow for new city.

12. TECH STACK & DATA SOURCES

12.1 Full Technology Stack

Category	Technology	Purpose	Cost
Frontend	React + Vite + TypeScript	Four-stage SPA. Sequential flow with stage progress indicator.	Free
Frontend	Tailwind CSS + Shadcn/UI	Styling and form components for preference intake.	Free
Frontend	Recharts	Future value projection line charts with best/worst case bands.	Free
Frontend	Mapbox GL JS	Optional neighborhood map with score heatmap overlay.	Free tier
Backend	FastAPI (Python)	REST API orchestrating all stages. Async parallel API calls.	Free
Backend	asyncio + httpx	Parallel external API calls (Walk Score, Google Places, Census).	Free
ML	scikit-learn	Random Forest classifier for neighborhood model. Preprocessing, cross-validation, GridSearchCV.	Free
ML	XGBoost	Regression model for future value projections (3 model instances).	Free
ML	pandas + numpy	Data cleaning, feature engineering, training dataset management.	Free
ML	joblib	Model serialization and loading for FastAPI serving.	Free
Data	FRED API (St. Louis Fed)	Real-time: 30-yr mortgage rate, Fed Funds Rate, Case-Shiller index, unemployment. Training: historical economic features.	Free
Data	Zillow Research CSVs	Training: ZHVI historical median price by zip (free download). Runtime: current CAGR and inventory levels.	Free
Data	Google Places API	POI counts and distances for neighborhood features.	~$0.003/call

Data	Walk Score API	Walkability, transit, bike scores.	~$0.002/call
Data	Census ACS API	Neighborhood demographics, income, population growth.	Free
Data	NCES Common Core	School location data and ratings.	Free
Data	Zillow Bridge / RapidAPI	Active home listings with all physical attributes.	~$0.01/call or free tier
Data	NAHB + RSMeans reference	Component lifecycle and replacement cost data for maintenance estimator.	Free reference
AI	Claude claude-sonnet-4-6 or GPT-4o	LLM Summary (1 call/session) and optional Future Cost Fallback (1 call/session).	~$0.02-0.04/call
Database	PostgreSQL	Caches neighborhood features, model scores, API responses.	Free
Testing	pytest	Unit, integration, LLM output rubric testing.	Free
Dev	Docker + docker-compose	Reproducible FastAPI + PostgreSQL environment.	Free
Dev	GitHub	Repository and project management.	Free

12.2 Data Sources by Stage

Stage	Data Source	What It Provides	Training vs Runtime
1	User input only	City, neighborhood preferences, house preferences	Runtime
2	Zillow Research ZHVI CSV	Zip-level historical median prices → CAGR feature + proxy labels	Both
2	Census ACS API	Demographics, income, population growth, commute patterns	Both
2	Walk Score API	Walkability, transit scores per neighborhood centroid	Both
2	Google Places API	Restaurant, park, hospital, grocery POI counts and distances	Both
2	NCES Common Core	School ratings by zip code	Both

2	FBI UCR / local crime data	Crime index per 1,000 residents by zip	Both
3	Zillow Bridge / RapidAPI	Active listing inventory in selected zip: price, beds, baths, sqft, type, year built, photos	Runtime only
4	FRED API	Current: mortgage rate, Fed Funds Rate, unemployment, CPI trend	Runtime only
4	FRED API (historical)	Historical economic indicators aligned with training transaction dates	Training only
4	County assessor records	Historical transaction pairs for regression model training	Training only
4	NAHB + RSMeans	Component lifecycle and replacement costs for maintenance estimator	Training (calibration)
4	LLM API	Summary generation + optional future cost fallback	Runtime only

12.3 Cost Estimate Per User Session

API Call	Estimated Cost	Notes
Walk Score API (per neighborhood, ~30 zips)	~$0.06	$0.002 x 30 neighborhoods. Cached after first session for that city.
Google Places API (per neighborhood, ~30 zips)	~$0.09	~1 call per zip for POI data. Cached.
Zillow listing fetch (one zip code)	~$0.05-0.10	Varies by provider. Cached per neighborhood.
FRED API	Free	Up to 500 requests/day at no cost.
LLM Summary call	~$0.02-0.04	~1,500 token input + 800 token output. Claude claude-sonnet-4-6 pricing.
LLM Fallback call (if active)	~$0.02-0.04	Same size as summary call. Only incurred if regression model not deployed.
ML model inference	Free	All models run locally on backend server.

TOTAL PER SESSION (cached city)	~$0.04-0.08	After first session for a city, most API calls hit cache.
TOTAL PER SESSION (cold start)	~$0.20-0.30	Full API calls for all 30 neighborhoods. Amortizes to near zero after caching.

12.4 First American Title Alignment

HomeScore serves the pre-transaction stage of the home buying journey — neighborhood selection and property evaluation before a buyer makes an offer. First American's products (title insurance, settlement services, closing tools) operate at and after the offer stage. HomeScore is therefore upstream of First American's existing funnel, not competitive with it. A buyer who uses HomeScore to identify and evaluate a property is a warmer, better-informed customer who is more likely to proceed confidently through closing — reducing transaction abandonment rates that directly affect First American's revenue.

The platform's maintenance cost projector also creates a natural adjacency to First American's home warranty products, which cover the exact systems (HVAC, plumbing, appliances, roof) that the estimator flags as high-risk. When the estimator flags a high-risk component, the UI could display a contextual First American home warranty CTA — a direct lead generation moment within an organic user journey. This transforms HomeScore from a standalone hackathon project into a demonstrable top-of-funnel product for First American's adjacent business lines.